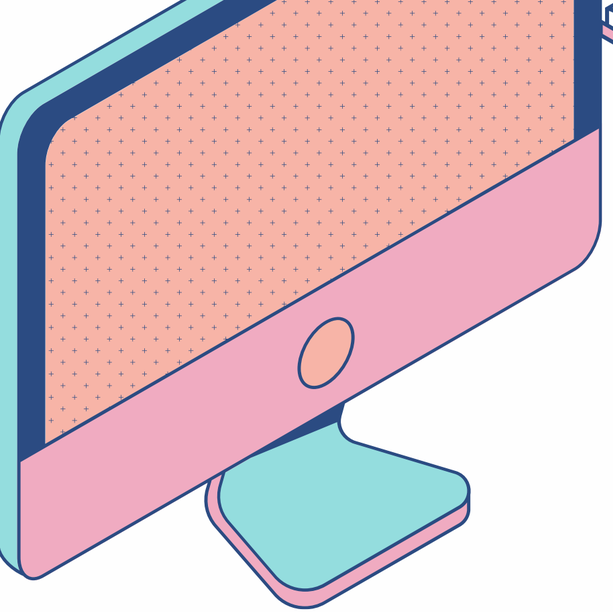
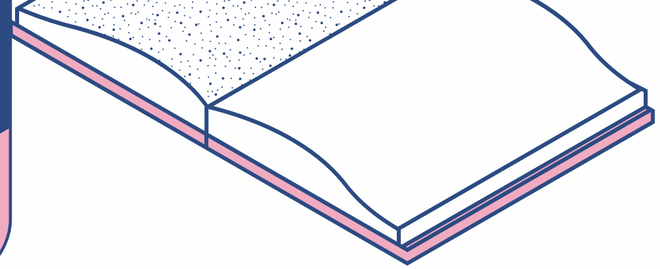
Analysis of Time complexity and handshaking logic

A Mini project on Computational Complexity

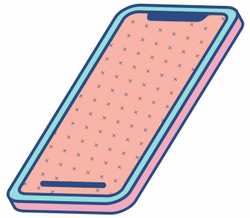
Course title : Discrete Mathematics
Course instructor : *AASR*
Section : 13
Group : 8 (Matrix Manipulators)
Group Members : *Zahedul* , *Diya* & *Adri*





INTRODUCTION

- Explanation the code
- Run the code
- Determine Edges ,Degrees & Handshaking logic hold
- Time complexity



```c
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
int adjMatrix[5000][5000]= {0};
int main(){
    int n;
    for(n=1000; n<=5000; n+=1000 ){
        int degrees[5000]= {0};
        int numOfedges = 0;
        int sumOfdegrees = 0;
        for(int i=0; i<n; i++){
            for(int j=i; j<n; j++){
                adjMatrix[i][j]=0;
                adjMatrix[j][i]=0;
            }
        }
        clock_t startTime= clock();
        printf("\nFor %d:\n",n);
        srand(time(NULL));
        for(int i=0; i<n; i++)
        {
            degrees[i]=0;
        }
```

```
numOfedges=0;
        for (int i =0; i<n; i++){
            for (int j=i+1; j<n; j++){

                if (rand() % 2){
                    adjMatrix[i][j] = 1;
                    adjMatrix[j][i] = 1;
                    degrees[i]++;
                    degrees[j]++;
                    numOfedges++;
                }
            }
        }

        for (int i=0; i<n; i++)
        {
            sumOfdegrees += degrees[i];
        }
```

```
        printf("Sum of degrees = %d\n", sumOfdegrees);
        printf("Number of edges = %d\n", 2 * numOfedges);
                if (sumOfdegrees == 2 * numOfedges){
            printf("Handshaking Logic holds.\n");
        }
        else
        {
            printf("Handshaking Logic does not hold.\n");
        }

        clock_t endTime= clock();

        double time = ((double)(endTime-startTime)/CLOCKS_PER_SEC)*1000.0;
        printf("Time = %f:\n", time);
    }
    return 0;
}
```

Output

```
For n = 1000:
sum_degrees = 499668
num_edges = 499668
Handshaking Logic holds.
Time = 17.000000:
```

```
For n = 2000:
sum_degrees = 1998900
num_edges = 1998900
Handshaking Logic holds.
Time = 60.000000:
```

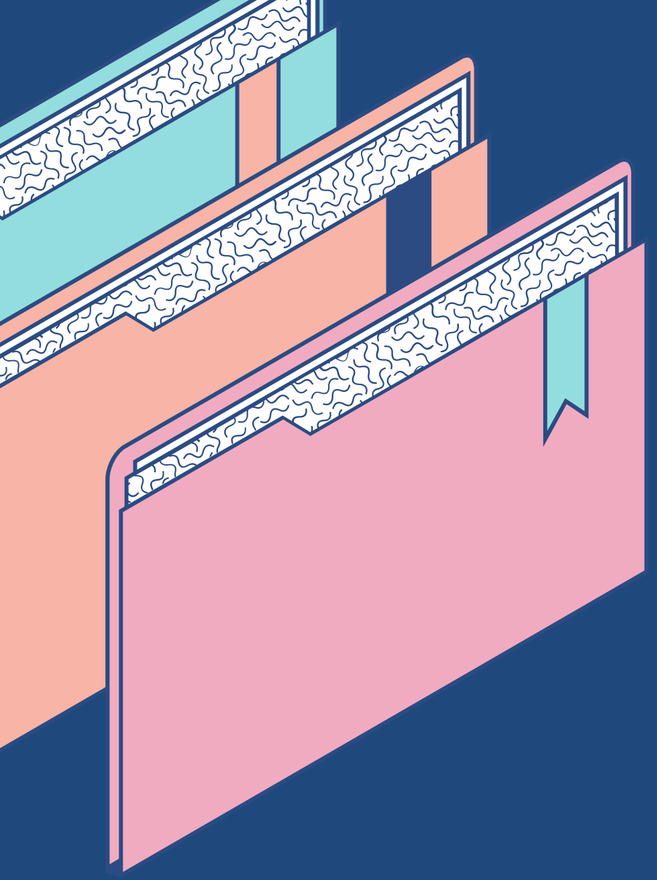
```
For n = 3000:
sum_degrees = 4498490
num_edges = 4498490
Handshaking Logic holds.
Time = 92.000000:
```

```
For n = 4000:
sum_degrees = 7998024
num_edges = 7998024
Handshaking Logic holds.
Time = 169.000000:
```

```
For n = 5000:
sum_degrees = 12497316
num_edges = 12497316
Handshaking Logic holds.
Time = 294.000000:
```

Counting Time ,Edges ,Degree & Handshaking logic hold

N of vertices	Computational time	Edges	Degree	Handshaking (Degree=2*edges)
1000	17	499668	499668	Logic hold
2000	60	1998900	1998900	Logic hold
3000	92	49984900	49984900	Logic hold
4000	169	7998024	7998024	Logic hold
5000	294	12497316	12497316	Logic hold



Theoretical

Time Complexity

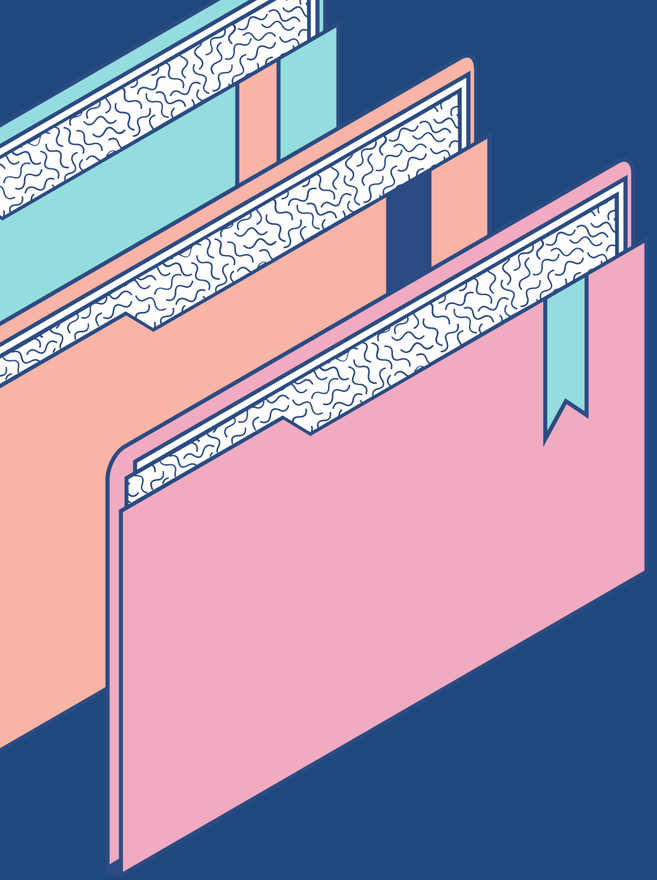
To analyze the theoretical time complexity of the given program,

1. **Graph Initialization:** Initializing an **n x n** adjacency matrix.
   Time Complexity : ***O(n^2)***

2. **Degree initialization:** Setting each element in the degree array to 0.
   Time Complexity: ***O(n)***

3. **Random Graph Generation**: Iterating over vertex pairs and updating the matrix and degrees.
   Time Complexity: ***O(n^2)***

4. **Degree Summation and Handshaking Logic Verification**: Summing degrees and checking the Handshaking Logic.
   Time complexity: ***O(n)***

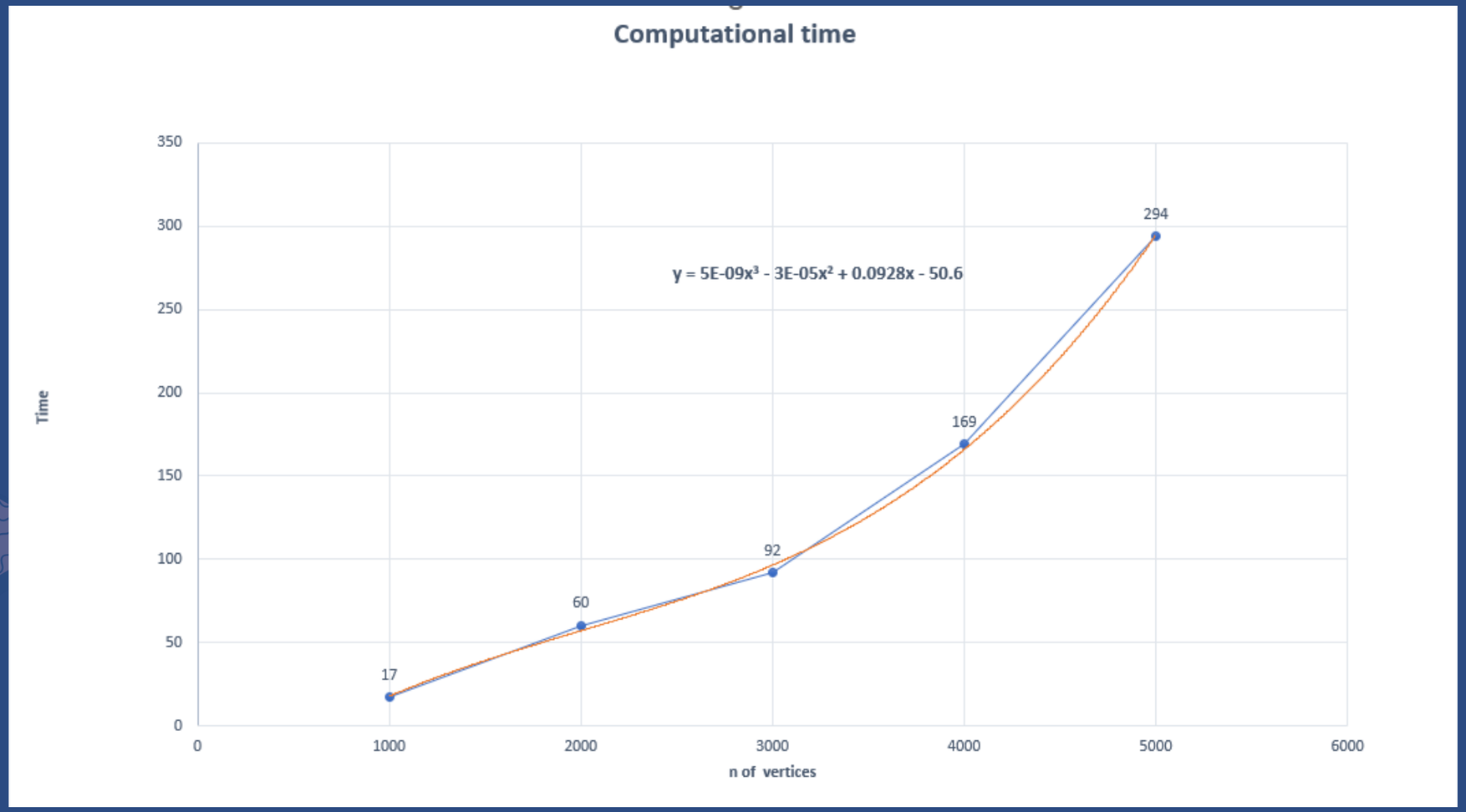


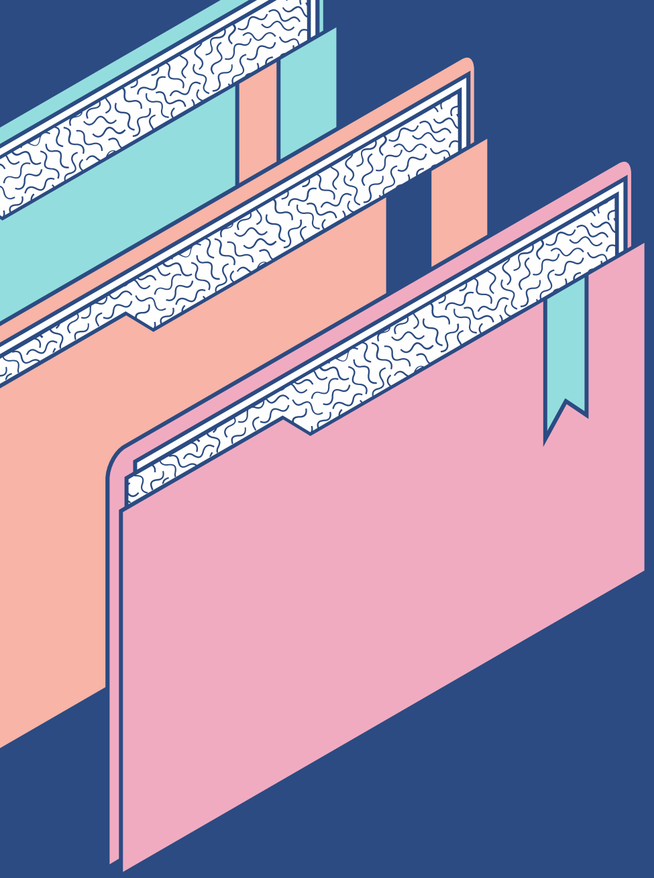
Combined Time Complexity

The dominant term is *O(n^2)* , making the overall theoretical time complexity.

O(n^2)

Computational time VS n line graph





Polynomial equation

The Time Complexity

y = 5E-09n^3 - 3E-05n^2 + 0.0928n - 50.6

-The given function ***y = 5E-09n^3 - 3E-05n^2 + 0.0928n - 50.6*** with Big-O notation, we need to focus on the term with the highest growth rate as n increases.

-The term with the highest degree ***5E-09n3*** dominates the behavior of the function as n grows large. The highest degree is ***n^3.***

-In Big-O notation, we can ignore constant ***50.6*** and lower degree terms ***3E-05n^2 & 0.0928n*** because they become insignificant compared to the highest degree term as n increases.

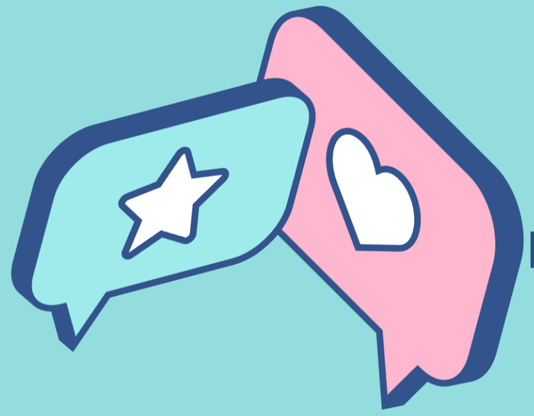
-Now, We can tell the time complexity of ***y = 5E-09n^3 - 3E-05n^2 + 0.0928n - 50.6*** as a function of n is ***O(n^3).***

Compare the theoretical and empirical Time complexity

Theoretical complexity, Summing the degrees and counting edges runs in *O(n^3)* time.

The empirical complexity from the Excel Graph also follow *O(n^3).*

--After comparison we can see both results are not same.



THANK YOU